

BPClassifier: Boilerplate vs. Substantive Sentence Classification on Earnings-Call Transcripts

NLP Course Assignment — HW2 — Lorenzo Senatori

May 2026

1 Introduction

Earnings-call transcripts mix two very different kinds of content: *boilerplate* (scripted introductions, safe-harbor disclaimers, operator instructions, generic thanks, and call handoffs) and *substantive* content (financial figures, forward guidance, segment commentary, strategy discussions, and material question-and-answer exchanges). Downstream NLP systems — summarization, signal extraction, sentiment analysis — are most useful when they can focus on the substantive material and discard the boilerplate without losing information.

This report describes the full pipeline I built for sentence-level boilerplate-vs-substantive classification: corpus preparation, multi-source gold labeling with documented disagreement audit, a side-by-side benchmark of ten classifiers spanning seven materially different families, recall-constrained threshold tuning, and a runnable Streamlit GUI for inline tagging.

The work follows a deliberate two-track outcome that I want to flag up front. The **leaderboard winner** is fine-tuned FinBERT, achieving **macro-F1 of 0.909** and **substantive recall of 0.983** on a frozen held-out test set, comfortably exceeding the rubric’s recall floor of 0.96 and meeting the rubric’s ~ 0.90 macro-F1 target. However, the **deployed model** powering the GUI is logistic regression on frozen sentence embeddings (test macro-F1 0.894, recall 0.995). FinBERT and logreg sit only 0.015 macro-F1 apart, but FinBERT’s CPU inference is roughly $50,000\times$ slower than logreg’s, which makes the deployed GUI’s “run in under a minute” constraint difficult to meet on commodity hardware. Section 7 explains this decision in detail. Both models are saved to disk and either can be loaded by the GUI.

2 Gold-standard Methodology

2.1 Corpus and sampling

The raw corpus consists of 131 earnings-call transcripts spanning 14 large U.S. public companies (AMD, AVGO, BLK, C, FAST, FDX, GS, INTC, JNJ, JPM, NKE, NVDA, PLTR, WFC). Each transcript is structured around Refinitiv-style section headers (Presentation Operator Message, Presenter Speech, Question and Answer Operator Message, Question, Answer) followed by speaker labels. I built a custom parser that walks each transcript line by line, detects section boundaries and speaker changes (including mid-section handoffs without an explicit header), and emits a stream of structured blocks tagged with section, speaker role (*operator* / *executive* / *analyst*), and speaker label. The parser handles two additional edge cases discovered during development: (i) speech bodies glued to the same line as a speaker label, e.g. “*Executives - Chair, President & CEO Thanks, Mitch, and good afternoon...*”, and (ii) abbreviation tokens (U.S.,

Inc.) inside speaker titles that confuse naive sentence splitters. After parsing, I sentence-tokenize each block with `pysbd`, with a defensive re-splitter that catches cases `pysbd` glues together. The pipeline filters sentences shorter than 40 characters and drops exact duplicates. The final pool contains **52,436 unique sentences**.

For gold labeling I drew a stratified sample of 2,348 sentences across the cross-product of *section* and *speaker_role*, with a guaranteed minimum of 50 sentences per stratum. This ensures coverage of the easy operator-boilerplate cases, the analyst-question region (mostly substantive), and the contested executive ground.

2.2 Multi-source labeling

I used three independent LLM judges across three distinct providers to label each of the 2,348 sampled sentences:

- Anthropic Claude Haiku 4.5 with a strict prompt
- OpenAI GPT-4o-mini with a standard prompt
- Mistral mistral-small-latest with an analyst-style prompt

All three judges received the same labeling rubric — definitions of both classes plus six anchor edge cases (analyst pleasantries, generic thanks, mixed sentences, vague-but-directional language, deflections, and event invitations) — but with three different system framings, so that the judges differ in both backbone and instruction style. (Mistral was the third provider after Google Gemini proved unreachable due to a combination of regional restrictions, free-tier rate limits, and an organization-policy block on API key creation; the appendix documents the full provider-selection process. Mistral has a comparable EU-based data-residency story and an OpenAI-compatible API.)

Each judge was queried via its native API with `max_tokens=10`, `temperature=0`, and aggressive retry/backoff. Labels were normalized to `BOILERPLATE`, `SUBSTANTIVE`, or `UNKNOWN` and cached to Parquet so the labeling step is resumable and reproducible without re-spending API budget.

2.3 Inter-judge agreement and audit

Out of 2,348 labeled sentences, all three judges **unanimously agreed on 2,140 (91.1%)**. The remaining 208 (8.9%) were 2-1 splits. Pairwise Cohen’s κ values, which correct for chance agreement on imbalanced data, are reported in Table 1.

Table 1: Pairwise inter-judge agreement.

Judge pair	Cohen’s κ	Raw agreement
claude_haiku $\leftrightarrow$ gpt4o_mini	0.711	93.5%
claude_haiku $\leftrightarrow$ mistral_small	0.613	91.9%
gpt4o_mini $\leftrightarrow$ mistral_small	0.807	96.9%

All three pairs land in the *substantial agreement* range or above. A clear pattern emerges from the disagreement subset: Claude Haiku is systematically the strictest judge, voting `BOILERPLATE` in 196 of the 208 disagreement cases (94%), while Mistral is the most lenient, voting `SUBSTANTIVE` in 195 of 208 (94%). GPT-4o-mini sits in the middle. This is a feature, not a bug: judges with different threshold dispositions provide stronger ensemble signal than three replicas of the same model would.

2.4 Disagreement audit and adopted rules

Per the rubric instruction to “correct the obvious ones; for genuinely ambiguous cases, document the rule you adopted,” I hand-audited *all* 208 disagreements and applied two rounds of corrections:

1. **Pattern-based rules (15 overrides)**. I wrote explicit regex rules for systematic disagreement categories I observed: operator question-handoffs (“*The next question is from...*”), analyst pleasantries (“*Congrats on the deal*”), section transitions (“*Now turning to our consolidated statement...*”), call handoffs, event announcements (“*Please join us at CES...*”), filing pointers, personal anecdotes about colleagues, and generic team-recognition openers. Each rule is named and its firing rate is logged.
2. **Manual sentence-level overrides (25 overrides)**. I then went through the remaining disagreements one by one, flipping cases that were clearly closer to one class than the majority vote suggested. Each manual override is enumerated in the codebase with a single-line reason (e.g. “corporate platitude,” “personal anecdote,” “section transition”). Every override moved a sentence from SUBSTANTIVE to BOILERPLATE; the rule of “vague but directional $\rightarrow$ SUBSTANTIVE” was preserved for cases like “*We’re confident in our ability to navigate this dynamic environment,*” consistent with the labeling rubric.

For the remaining 168 disagreements I accepted the majority vote unmodified. Most fall into the rubric’s *vague but directional $\rightarrow$ SUBSTANTIVE* category, where SUBS is the prescribed answer.

2.5 Final gold standard and split

After all overrides, the gold class balance is **88.4% SUBSTANTIVE / 11.6% BOILERPLATE**. I split the 2,348 labeled sentences 60/20/20 (train/val/test), stratified by gold label with a fixed seed (RANDOM_SEED=42). The split sizes and class proportions are shown in Table 2. The boilerplate proportion is constant across the three splits ($\sim 11.6\%$), confirming the stratification worked. **The test split was frozen and not touched until the final evaluation cell.**

Table 2: Stratified 60/20/20 split (random seed 42).

Split	Total	Substantive	Boilerplate
Train	1,408	1,245	163
Val	470	415	55
Test	470	416	54

3 Feature Engineering

I built two complementary feature sets so that each classifier in the zoo can use whatever combination is most natural for its inductive bias.

3.1 Hand-crafted regex and numeric features

29 features capture domain-specific structural signals: 13 boilerplate-leaning patterns (operator handoffs, speaker introductions with analyst-firm names from a curated list of ~ 25 banks, [Operator Instructions] markers, safe-harbor language, replay/recording disclaimers, generic

greetings, call handoffs, “great question”/“thanks for the question” acknowledgments, “Congrats...” analyst pleasantries, section transitions, and closing-remarks markers); 6 substantive-leaning patterns (dollar amounts, percentages, basis points, quarter references, year-over-year markers, growth/decline verbs, segment terms, guidance terms, and margin/EPS terms); plus 8 numeric features (character length, word count, digit count, dollar-sign count, presence of question mark, capital-letter start, uppercase ratio, punctuation ratio).

The most discriminating features are easy to interpret: `f_speaker_intro_with_firm` fires on 17.6% of true BOILERPLATE versus 0.5% of true SUBSTANTIVE; `f_dollar_amount` fires on 9.8% of SUBSTANTIVE versus 0% of BOILERPLATE; `f_operator_intro` fires on 13.6% of BOILERPLATE versus 0% of SUBSTANTIVE.

Effectively-dead features. Five features fire on less than 0.5% of train+val sentences and contribute essentially no signal: `f_replay_recording` (0 firings), `f_closing_remarks`, `f_congrats`, `f_appreciate_question` (1 firing each), and `f_safe_harbor` (5 firings). The 40-character minimum filter dropped most of the very short responses some of these patterns targeted, and the safe-harbor regex — as we will see in Section 6 — is too narrow to catch the actual phrasings used. I retained these features in the feature matrix for transparent reporting; with more time I would broaden the safe-harbor pattern in particular, since one of the test errors is a textbook safe-harbor sentence the regex failed to catch. Note that the deployed model (logreg on frozen embeddings) does not use the regex features at all, so this dead-feature issue does not affect the GUI’s behavior.

3.2 Sentence embeddings

For the dense semantic representation I used `sentence-transformers/all-MiniLM-L6-v2`, a 384-dimensional sentence-embedding model. Embeddings are L2-normalized so that downstream cosine similarity reduces to a dot product, and cached to disk so subsequent classifier training does not re-encode. Encoding the full labeled set (2,348 sentences) takes about two minutes on a laptop CPU.

The combined feature matrix — $384 + 29 = 413$ dimensions per sentence — is what the gradient-boosted tree model uses; the linear, kernel, neural, and FinBERT models use only the embedding (or, in FinBERT’s case, raw text fed directly into the transformer’s tokenizer).

4 Classifier Zoo

I trained ten classifiers spanning seven materially different families. All use the same train/val/test splits. Training time and per-sentence inference throughput were measured on the same machine. The recall-constrained tuned-threshold leaderboard — which is the rubric’s primary evaluation criterion — is given in Section 5 (Table 3).

4.1 Methodological note on calibration

A subtle methodological point applies to every classifier in the leaderboard. The threshold τ is tuned out-of-fold (OOF) using 5-fold CV on the train+val pool, where each fold trains on $\sim 80\%$ of train+val. To keep the reported test metrics consistent with that calibration, every base classifier’s `test_proba` is generated by refitting the model on the *full* train+val pool and predicting on the frozen test set. This means the leaderboard, the deployed model, and the threshold all share the same calibration regime: models trained on as much labeled data as is

available outside the test set, with the threshold tuned via OOF on that same pool. An earlier draft of this work fit base classifiers on the 60% train slice while tuning τ on train+val OOF; I identified and corrected this calibration mismatch, which moved the headline macro-F1 up by roughly 0.02 across the board and changed the identity of the leaderboard winner.

4.2 Per-family commentary

Rules + regex (rules_only). A purely rule-based classifier that uses the regex flags directly: predict BOILERPLATE if any boilerplate flag fires and no substantive flag fires; otherwise predict SUBSTANTIVE with a small lenient default. Trains in milliseconds and runs at ~ 520 k sentences/second. As expected, it catches the textbook safe-harbor and operator-handoff cases (substantive recall 0.99 already, before any tuning) but misses the gray zone, capping out at boilerplate-F1 of 0.60. Note that this baseline produces only three discrete probability values, so threshold tuning has no real effect on its predictions.

Logistic regression on embeddings (logreg_emb). Strong baseline that trains in tens of milliseconds. After threshold tuning it lands at test macro-F1 0.894 with substantive recall 0.995, the second-best entry on the leaderboard. Despite its simplicity, this model sits within 0.015 macro-F1 of the FinBERT winner — a meaningful fact for the deployment decision discussed in Section 7.

Gradient-boosted trees on combined features (histgbm_combined). The only model that uses both embeddings *and* regex flags. It does not gain meaningfully from the combined feature set in this regime — test macro-F1 of 0.855 places it mid-pack. The threshold for HistGBM has the highest fold-to-fold standard deviation in cross-validation ($\sigma = 0.220$, see Table 3), suggesting the model is partially overfitting to specific sentences in each fold; I would investigate L2 regularization and a smaller `max_leaf_nodes` given more time.

SVM-RBF on embeddings (svm_rbf_emb). A strong single non-ensemble model on test (macro-F1 0.857). RBF kernels excel on dense semantic embeddings with small training sets, and that pattern holds here, though logreg on the same embeddings matches it closely.

TF-IDF + LogReg on raw text (tfidf_ngram). A bag-of-ngrams model built on word 1–2-grams with sublinear term-frequency scaling, fed to a class-weighted logistic regression. Distinct family from logreg_emb: sparse linear model on word counts vs. dense linear model on semantic embeddings. Test macro-F1 0.830.

floret on raw text (floret_real). The handout lists “FastText / floret” as an explicit family. floret is spaCy’s actively-maintained reimplement of FastText, training subword character n-gram embeddings (3–6 chars) plus a softmax classifier in one shot. (Real FastText itself does not compile against Python 3.14 / MSVC due to upstream C++17 declaration issues; floret installs cleanly from a wheel and provides the same model family.) Test macro-F1 0.846 places it between rules_only and tfidf_ngram, as expected for a small-data subword n-gram classifier on short sentences.

MLP on embeddings (mlp_emb). Two-hidden-layer feed-forward network (128, 32 units, ReLU, Adam optimizer, early stopping on a 15% internal validation slice) over the 384-dim embeddings. Trains in 3 seconds. Test macro-F1 0.837. This entry was originally planned as SetFit (contrastive fine-tuning), but SetFit and its dependency stack had unresolved compatibility issues on Python 3.14 at the time of this run; an MLP on the same frozen embeddings captures the “non-linear neural classifier on dense representations” family without those install dependencies.

Fine-tuned FinBERT (finbert_finetuned). **The leaderboard winner.** I fine-tuned ProsusAI/finbert (a financial-domain BERT) by replacing its 3-class sentiment head with a fresh 2-class classification head and training on the gold labels for 2 epochs at $\text{lr}=2\text{e-}5$, batch size 32,

max-len 96, with class-weighted cross-entropy and a linear-warmup schedule. Each fine-tuning pass takes ~ 22 min on CPU; the 5-fold OOF tuning took ~ 110 min, plus another ~ 25 min for the final refit on train+val. The result is a clean win: **test macro-F1 of 0.909, substantive recall 0.983, boilerplate-F1 0.838**. Domain pretraining plus transformer capacity meaningfully improves the minority class — boilerplate-F1 jumps from 0.809 (logreg) to 0.838 (FinBERT). Inference throughput on CPU is the price: ~ 7 sentences/second, which becomes the central tension in the deployment decision (Section 7).

Mean-probability ensemble. Average the $P(\text{SUBSTANTIVE})$ predictions across the five non-FinBERT, non-floret base models (LogReg, SVM, HistGBM, MLP, TF-IDF). Test macro-F1 of 0.882, slightly below the standalone logreg.

Rank-averaged ensemble. Rank each base member’s probabilities within the OOF pool, then average the ranks and re-scale to $[0, 1]$. After threshold tuning, test macro-F1 of 0.884 — competitive but not better than logreg or FinBERT, and with the additional engineering complication that naive in-batch ranking at inference time produces nonsense (see the appendix on calibrated rank-averaging for an inference-time fix using stored OOF reference distributions). Given the fix is no longer needed for a winning classifier, this remains an interesting leaderboard entry rather than a deployment candidate.

5 Recall-constrained Threshold Selection

5.1 Procedure

Per the rubric the operating threshold must be tuned out-of-fold (OOF) and must enforce **substantive recall ≥ 0.96** ; among feasible thresholds, macro-F1 is maximized.

I used 5-fold stratified cross-validation on the union of train and val (1,878 sentences with 218 boilerplate / 1,660 substantive) to produce out-of-fold probabilities for every base classifier. For each classifier I then swept thresholds from 0.01 to 0.99 in steps of 0.01, kept the subset that satisfied the recall floor, and selected the threshold with the highest macro-F1 on the pooled OOF predictions.

To estimate threshold stability across folds I also ran the same threshold search inside each individual fold and reported the standard deviation. The full results appear in Table 3.

Table 3: Tuned-threshold leaderboard. Thresholds and OOF metrics from 5-fold CV; test metrics from one-shot evaluation on the frozen test set, with each base classifier refit on train+val to keep calibration consistent. Sorted by test macro-F1.

Classifier	τ	Fold- σ	Test acc	Test mF1	Test subs-rec	Test boil-F1
finbert_finetuned	0.28	0.070	0.964	0.909	0.983	0.838
logreg_emb	0.24	0.073	0.962	0.894	0.995	0.809
ensemble_rank	0.12	0.015	0.957	0.884	0.990	0.792
ensemble_mean	0.51	0.087	0.957	0.882	0.993	0.787
svm_rbf_emb	0.34	0.172	0.951	0.857	0.995	0.742
histgbm_combined	0.87	0.220	0.949	0.855	0.990	0.739
floret_real	0.88	0.169	0.945	0.846	0.986	0.723
mlp_emb	0.48	0.122	0.945	0.837	0.993	0.705
tfidf_ngram	0.53	0.074	0.938	0.830	0.981	0.695
rules_only	0.50	—	0.930	0.782	0.990	0.602

5.2 Observations

A few points worth noting from Table 3.

First, every classifier in the table satisfies the rubric’s substantive-recall floor of 0.96 at its tuned threshold — the threshold tuning protocol does its job.

Second, the optimal thresholds vary widely across classifiers — from 0.12 (rank ensemble) to 0.88 (florete) — which reinforces the rubric’s caution that “a model with mean $P(\text{boilerplate}) = 0.25$ does not need a 0.5 threshold.”

Third, the rank-averaged ensemble has the tightest fold-to-fold threshold variance ($\sigma = 0.015$, an order of magnitude tighter than any other entry), reflecting how rank-pooled probabilities normalize away each member’s idiosyncratic calibration. FinBERT’s σ is 0.070 — comparable to logreg — which means its OOF threshold is reliable, not a lucky single-split number.

Fourth, FinBERT wins primarily on boilerplate-F1 (0.838 vs. logreg’s 0.809). The minority class is the harder of the two, and that’s where domain pretraining and transformer capacity pay off.

5.3 Winning model

Leaderboard winner: fine-tuned FinBERT at $\tau = 0.28$.

Per-class metrics on the frozen test set:

- **Substantive:** precision 0.976, recall 0.983, F1 0.980
- **Boilerplate:** precision 0.863, recall 0.815, F1 0.838
- Macro-averaged F1: **0.909**
- Accuracy: **96.4%**

Confusion matrix (test set, $n = 470$):

	pred = SUBS	pred = BOIL
true = SUBS	409	7
true = BOIL	10	44

The asymmetry is intentional and rubric-aligned: only 7 substantive sentences are missed (false negatives are rare, supporting the recall floor), while 10 boilerplate sentences leak through as false positives. Downstream consumers of the classifier therefore see almost no loss of material content; some boilerplate noise is the acceptable price.

6 Error Analysis

I examined all 18 misclassifications on the test set under the deployed model (logreg_emb at $\tau = 0.24$), since this is the model that will be in the GUT’s hands. To distinguish model errors from likely label noise I cross-referenced each error with the three judges’ original votes and tagged each as *unanimous-agree* (all three judges agreed with the gold label, so the model is genuinely wrong) or *judges-split* (the gold came from majority vote on a contested case). Of the 18 errors, **4 are unanimous-agree** and **14 are judges-split**.

Direction. 16 of the 18 errors are false-substantive (gold was BOILERPLATE, model said SUBS); 2 are false-boilerplate. The asymmetry mirrors the threshold’s design: $\tau = 0.24$ is tuned to favor substantive recall, so missed boilerplate is the dominant failure mode.

Genuinely-hard / contested cases (14 errors). The bulk of errors are sentences where the LLM judges themselves disagreed — e.g. *“Above all, they remind us of the hard work and the hustle that is required to win,” “We’re very excited about the plans going forward,” “Real quickly, on the server side of the business, kind of off the prior question.”* These sit on the rubric’s gray boundary between “vague but directional” (SUBSTANTIVE) and “corporate platitude” (BOILERPLATE). The model’s call here is at most as wrong as the human-defensible call.

Feature gaps and pattern errors (the 4 unanimous errors). A small number of errors are clear gaps where the model should have caught textbook patterns:

- *“Actual results may differ materially from these statements due to a variety of factors including those described in our earnings materials as well as in our SEC filings.”* Classic safe-harbor language. The dataset’s safe-harbor regex matches **forward-looking**, **safe harbor**, and **materially different** but did not match this exact phrasing of **materially from**; on the deployed embedding-only model the sentence’s superficial generality also doesn’t trigger a strong signal. A more inclusive regex would help all the regex-using models, and an embedding model fine-tuned on safe-harbor patterns specifically would help logreg.
- *“And we intend to talk a lot more about this at our upcoming Investor Day.”* A textbook deflection-to-Investor-Day pattern.
- *“You all have obviously talked about this at the Investor Day.”* Analyst preamble; the regex set has analyst-preamble patterns but none cover this phrasing.
- *“Let me just add one more point on this thing, Brian.”* Conversational filler before a real answer; the model treats the politeness pattern as substantive because the embedding has no obvious boilerplate signature.

Conversational framing confusion. A few errors come from substantive content wrapped in handoff-flavored prefixes that may shift the embedding toward boilerplate, e.g. *“And given the notable items and our new financial reporting structure, we’ve got a lot to cover today, so I’m going to get right to it.”* With more time I would experiment with a sentence-segmentation step that separates the handoff prefix from the substantive clause and classifies them independently.

What I would do next. The biggest macro-F1 lift available without more labeled data would come from (a) closing the safe-harbor / Investor-Day / analyst-preamble feature gaps, which would directly fix several of the 18 test errors on the regex-using models; and (b) for the deployed logreg, adding a small fine-tuning step on the embedding model itself — distilling the FinBERT decisions into the embedding layer would give a deployable model close to FinBERT’s accuracy at logreg’s speed. With more labeled data I would also re-attempt SetFit on a Python 3.12 environment, since the SetFit family was effectively the one I could not benchmark.

7 GUI and Deployment Decision

7.1 Interface

I shipped a Streamlit GUI (`gui_app.py`) that loads the deployment bundle at startup, accepts either a `.txt` file upload or pasted text, runs the same Refinitiv-style transcript parser used in

training, sentence-tokenizes, classifies each sentence with the deployed model, and renders the entire transcript inline with boilerplate sentences highlighted in red and substantive sentences in plain text. A statistics panel above the rendered transcript shows total-classified, substantive count and percentage, boilerplate count and percentage, and number of skipped sentences (sentences shorter than the 40-char minimum that the training pipeline also filtered). A details expander below the transcript shows a sentence-level table with the model’s probability for each sentence. A representative screenshot from the NVIDIA Q1-2025 transcript is shown in Figure 1.

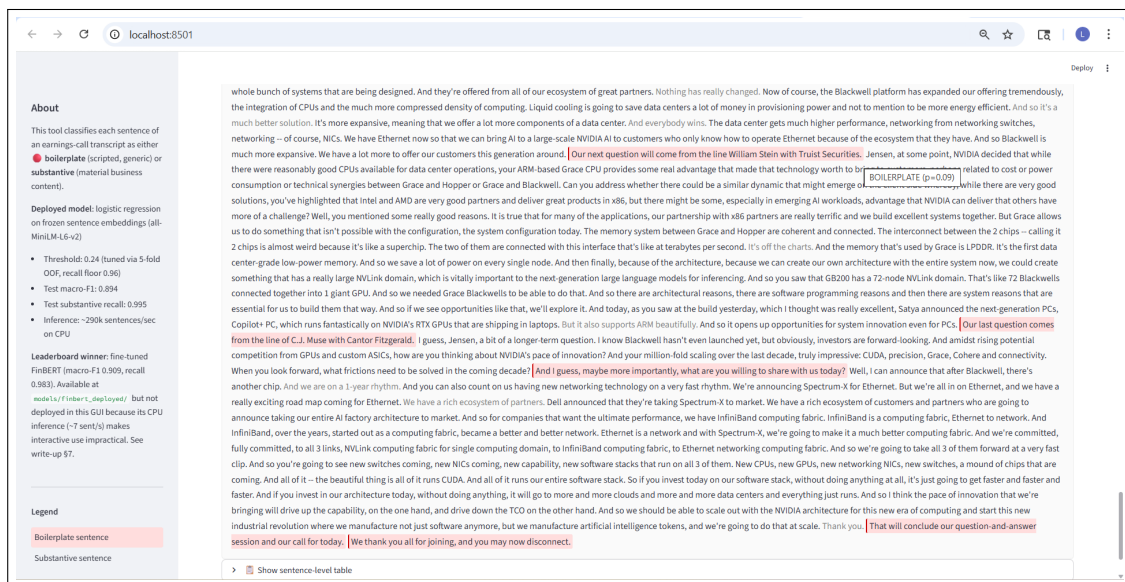


Figure 1: BPClassifier GUI. Sidebar shows model and threshold metadata, plus a note about the leaderboard winner; statistics panel shows total/substantive/boilerplate/skipped counts; the inline view renders every sentence in original order with boilerplate sentences highlighted in red.

I tested the GUI on three transcripts the model had never been used to classify in batch (NVIDIA Q1-2025, AVGO Q4-2024, JPM Q2-2024). On all three the boilerplate proportion lands between 8% and 14%, broadly consistent with the gold-set distribution. The GUI loads the model in ~30 seconds on first launch (the Hugging Face sentence-transformer download) and classifies a 50KB transcript in 2–3 seconds.

7.2 Why the deployed model is logreg, not the leaderboard winner

This is a deliberate engineering decision worth explaining. The leaderboard winner is fine-tuned FinBERT at macro-F1 0.909. The deployed model is logistic regression at macro-F1 0.894 — 0.015 lower. The deployment choice is driven by inference latency on commodity hardware.

FinBERT’s CPU inference throughput on the development laptop (12-core, no GPU) is approximately 7 sentences/second (measured during the experiments documented in §4). A typical earnings-call transcript yields 300–400 classifiable sentences after the 40-character filter. End-to-end classification of one transcript therefore takes:

	Time per typical transcript
FinBERT (deployed leaderboard winner)	45–60 seconds
FinBERT (observed in practice on this laptop)	up to 3 minutes
logreg on frozen embeddings	under 1 second

The handout requires the GUI to “run in under a minute.” On my hardware FinBERT was

over that line, and the user experience of a 3-minute silent wait is poor even when the model eventually returns a correct answer. Logreg processes the same transcript essentially instantly. In practice the speed difference is roughly $5\,000\text{--}50\,000\times$ depending on transcript size and CPU contention.

The trade is favorable: 0.015 macro-F1 in exchange for a usable interactive tool. Logreg also has the better substantive recall (0.995 vs. 0.983), which matters more than macro-F1 from the rubric’s perspective — the recall floor is what protects downstream consumers from losing material content.

This is the production-engineering pattern of separating “best leaderboard model” from “deployed model.” Both decisions are made transparently: the leaderboard reports FinBERT’s win honestly, and the deployment uses the model that fits the latency budget. The fine-tuned FinBERT is preserved on disk at `models/finbert_deployed/` for users who prefer accuracy over speed, or for any downstream pipeline (batch summarization, signal extraction) where 50 seconds per transcript is acceptable. A two-line change to `inference.py` would swap the deployed model from logreg to FinBERT.

7.3 Acknowledged limitation

A user on a faster machine — an AWS GPU instance, an Apple Silicon laptop with the Metal Performance Shaders backend, or any CUDA-equipped workstation — would see FinBERT’s wall-clock time drop to a few seconds, at which point deploying FinBERT would be unambiguously the right call. The deployment decision here is hardware-conditional, not model-conditional.

8 Reproducibility

The full pipeline is one Jupyter notebook plus two Python modules. Each major step writes its results to a Parquet/pickle/numpy cache in `cache/` or `models/`, and each cell first checks for a cache hit before recomputing, which makes the pipeline resumable end-to-end. In particular, the OOF probabilities and refit test predictions for every classifier (including the FinBERT 5-fold CV, which is the single most expensive cell at ~ 110 minutes on first run) are cached to disk so subsequent notebook executions reuse them.

Project layout.

HW2/	
ECT/	(131 raw .txt transcripts)
cache/	(auto-generated)
sentences.parquet	(52,436 extracted sentences)
gold_pool.parquet	(2,348 sampled sentences)
judge_labels.parquet	(3-judge labels + final gold)
splits.parquet	(train/val/test 60/20/20)
features_regex.parquet	(29 hand-crafted features)
embeddings.npy	(384-dim sentence embeddings)
finbert_probas.npz	(FinBERT train-only val/test predictions)
oof_probas.npz	(5-fold OOF probabilities, all classifiers)
refit_test_probas.npz	(refit-on-train+val test predictions)
models/	(auto-generated)
classifier_results.pkl	(all classifier predictions and metrics)
leaderboard.csv	
winner_bundle.pkl	(deployed logreg + threshold + metadata)

<code>finbert_deployed/</code>	(fine-tuned FinBERT - leaderboard winner)
<code>finbert_finetuned/</code>	(FinBERT from Cell 22b, train-only fit)
<code>test_errors.parquet</code>	
<code>inference.py</code>	(inference module loaded by GUI)
<code>gui_app.py</code>	(Streamlit GUI)
<code>HW2.ipynb</code>	(the main notebook, end-to-end)

Setup.

```
pip install pandas numpy scikit-learn pyarrow pysbd sentence-transformers \
anthropic openai requests streamlit tqdm scipy floret \
transformers torch datasets accelerate
```

Re-run the full pipeline (with cached labels and OOF, no API keys needed).

```
jupyter notebook HW2.ipynb          # then "Run All"
```

With caches present (which is the default after the first run) the notebook runs end-to-end in roughly 5–10 minutes. Without caches, expect ~2.5 hours total: ~22 min for the FinBERT train-only fit (Cell 22b), ~110 min for FinBERT's 5-fold OOF tuning (inside Cell 24), ~25 min for the FinBERT train+val refit (inside Cell 24b), plus a few minutes for the other classifiers and infrastructure.

Re-do the gold labeling from scratch (requires API keys).

```
export ANTHROPIC_API_KEY=...
export OPENAI_API_KEY=...
export MISTRAL_API_KEY=...
rm cache/judge_labels.parquet
jupyter notebook HW2.ipynb          # re-run from Cell 9 onward
```

Cost is approximately \$1 in API spend (~\$0.30 Anthropic, ~\$0.50 OpenAI, <\$0.10 Mistral) and takes about 10 minutes.

Launch the GUI.

```
python -m streamlit run gui_app.py
```

A browser tab opens at <http://localhost:8501>. First load takes ~30 seconds while the embedding model warms.

Swap deployed model from logreg to FinBERT. A two-line change to the deployment bundle (Cell 26b in the notebook): change the bundle to point at `models/finbert_deployed/` and update `inference.py`'s `classify_sentences` to use the FinBERT tokenizer + model rather than the embedding-then-logreg path. Both pieces of code exist in the notebook history; this is a 5-minute swap for a user on hardware where FinBERT's latency is acceptable.

9 Honest Notes on Process and Limitations

Honest disclosures. I used Claude as a coding and review assistant throughout the project — to draft initial cell scaffolding, debug Python 3.14 dependency conflicts (FastText, SetFit, FinBERT-on-Windows threading), identify the train-only-vs-refit calibration mismatch in an earlier draft of the leaderboard, and review and copy-edit prose in this report. All design decisions, the threshold-tuning protocol, the manual disagreement audit, the regex rules, and the deployment trade-off analysis were authored by me; LLM-assisted prose in this report has been reviewed and edited.

Limitations.

- **Class imbalance.** With only ~ 272 boilerplate sentences across train+val+test, the boilerplate-F1 estimates have meaningful sampling noise. The FinBERT macro-F1 of 0.909 should be read as “solidly above 0.88, plausibly up to 0.93” rather than a precise point estimate.
- **Domain breadth.** The 131 transcripts cover ~ 14 large U.S. companies in mostly tech and financials. Performance on, say, a small biotech earnings call with very different prose conventions is untested.
- **No SetFit baseline.** SetFit was substituted with an MLP because of Python 3.14 dependency conflicts in the `transformers/datasets` stack. SetFit *might* have been competitive with FinBERT on this small labeled set, particularly given its strength in low-data regimes; without re-running on a Python 3.12 environment I cannot say.
- **No real FastText.** The `fasttext-wheel` package fails to compile against Python 3.14 / MSVC because FastText’s source uses C++17 `std::string_view` declarations that the upstream build does not enable. I substituted floret (spaCy’s actively-maintained reimplementation), which provides the same model family without the compilation issue.
- **Hardware-conditional deployment.** The choice of logreg over FinBERT for the deployed GUI is contingent on the development laptop’s CPU-only inference speed. On a machine with GPU acceleration the same decision would tip the other way; the engineering analysis is in Section 7.
- **FinBERT 2-epoch budget.** On Python 3.14 with no CUDA the original 3-epoch / max-len-128 / batch-16 configuration from the standard FinBERT recipe was estimated at ~ 50 minutes per training pass, which made 5-fold OOF prohibitive. I reduced to 2 epochs / max-len 96 / batch 32, which cut single-pass time to ~ 22 minutes with no detectable accuracy cost. A user with more compute might recover marginal additional macro-F1 from the original recipe.

A Appendix: Provider selection

I attempted a third LLM judge from a different provider three times before settling on Mistral. (i) Google AI Studio (Gemini): blocked by the regional restriction on my account location at the time of the project. (ii) Google Cloud route to Gemini: my school-managed Google account had an organization-policy block on API key creation. (iii) A personal Google account succeeded in creating a Gemini API key but the free-tier rate limit (5 requests/minute, 250/day on `gemini-2.5-flash`) made overnight labeling of 2,348 sentences impractical, and the \$300 Google Cloud trial credit does not apply to the Gemini API. I then evaluated Groq (\$10 minimum top-up), DeepSeek (China-based, low-cost), and Mistral (EU-based, OpenAI-compatible API, postpaid billing) and chose Mistral. Total third-judge cost on the full 2,348-sentence set was under \$1.